Generatore di frattali di Lyapunov parallelizzato

Tommaso Raffaelli

2026-03-22

Contents

1 Introduzione **2**

2 Frattali **2**

2.1 Frattalli di Lyapunov 3

2.2 Algoritmo 3

3 OpenMP **3**

3.1 Scheduling 6

3.2 Sublinearita' 9

1 Introduzione

L'obbiettivo del progetto e' quello di esplorare l'ottimizzazione, per il tempo di esecuzione, di un programma sequenziale attraverso l'utilizzo delle direttive di OpenMP cercando di dimostrare la sublinearita' del programma parallelizzato. In genere il calcolo di frattali si prestano molto bene alla parallelizzazione dato che sono una serie molto lunga di calcoli indipendenti fra di loro e generalmente anche costosi a livello computazionale.

In particolare verranno usati i frattali di Lyapunov che e' una rappresentazione visiva della stabilita' di sistemi dinamici non lineari, la cui generazione richiede il calcolo dell'esponente di Lyapunvo per ogniuna delle coordinate all'interno della griglia bidimensionale usata per la rappresentazione del frattale.

2 Frattali

Un frattale e', intuitivamente, una figura in cui un singolo motivo viene ripetuto su scale descrescenti. Come si può vedere osservando la curva di Von Koch:

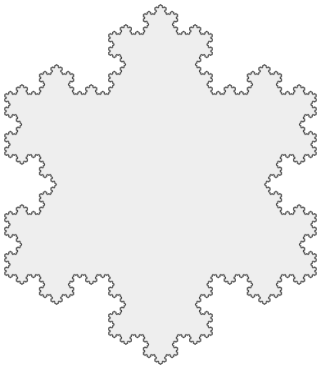


Figure 1: Curva di Von Koch

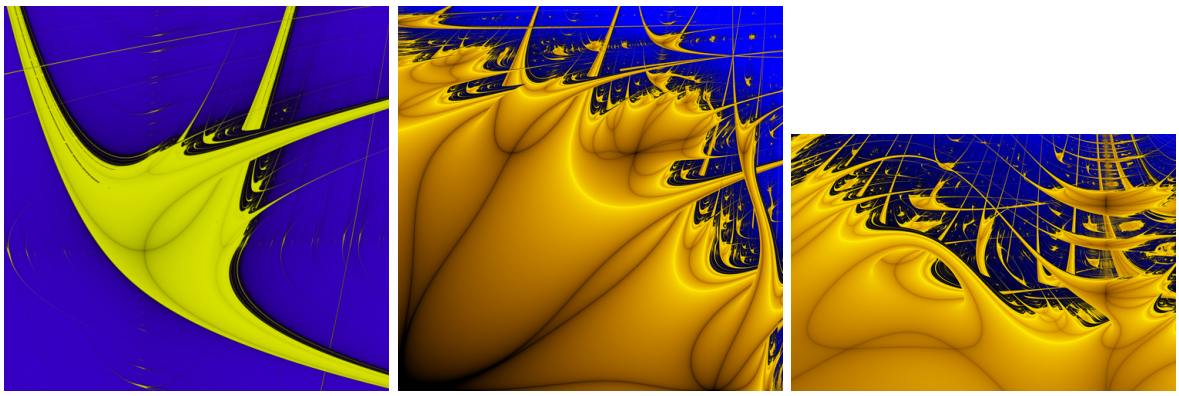
Si considera frattale un insieme che goda di tutte o molte delle seguenti proprietà:

- **Autosimilarita'**: il frattale e' unione di copie di se stesso a scale differenti
- **Struttura simile**: il frattale rileva dettagli ad ogni ingrandimento
- **Irregolarita'**: il frattale non puo' essere descritto come luogo dei punti che soddisfano semplici condizioni geometriche o analitiche
- **Dimensione non intera**: sebbene un frattale possa essere rappresentato in uno spazio convenzionale a due o tre dimensioni

2.1 Frattalli di Lyapunov

In matematica i frattali di Lyaounov sono frattali biforcativi derivati da un'estensione della mappa logistica in cui il grado di crescita della popolazione, r, commuta periodicamente tra due valori A e B.

Un frattale di Lyapunov e' costruito tramite la mappatura delle regioni di stabilita' e chaos, calcolati tramite l'esponente di Lyapunov λ, le zone gialle corrispondono a $\lambda < 0$ (stabilita') e quelle blu corrispondono a $\lambda < 0$ (chaos).



2.2 Algoritmo

1. Scegli una stringa di A e B di qualsiasi lunghezza
2. Costruisci la sequenza S formata dalla stringa prima definita, ripetuta quante volte servisse
3. Scegli un punto $(a, b) \in [0, 4] \times [0, 4]$
4. Definisci la funzione $r_n = a$ se $S_n = A$, e $r_n = b$ se $S_n = B$
5. Partendo da $x_0 = 0.5$ calcola le iterazioni $x_{n+1} = r_n x_n (1 - x_n)$
6. Calcola l'esponente di Lyapunov

$$\lambda = \lim_{N \to \infty} \frac{1}{N} \sum_{n=1}^{N} \log \left| \frac{dx_{n+1}}{dx_n} \right| = \lim_{N \to \infty} \frac{1}{N} \sum_{n=1}^{N} \log |r_n (1 - 2x_n)|$$

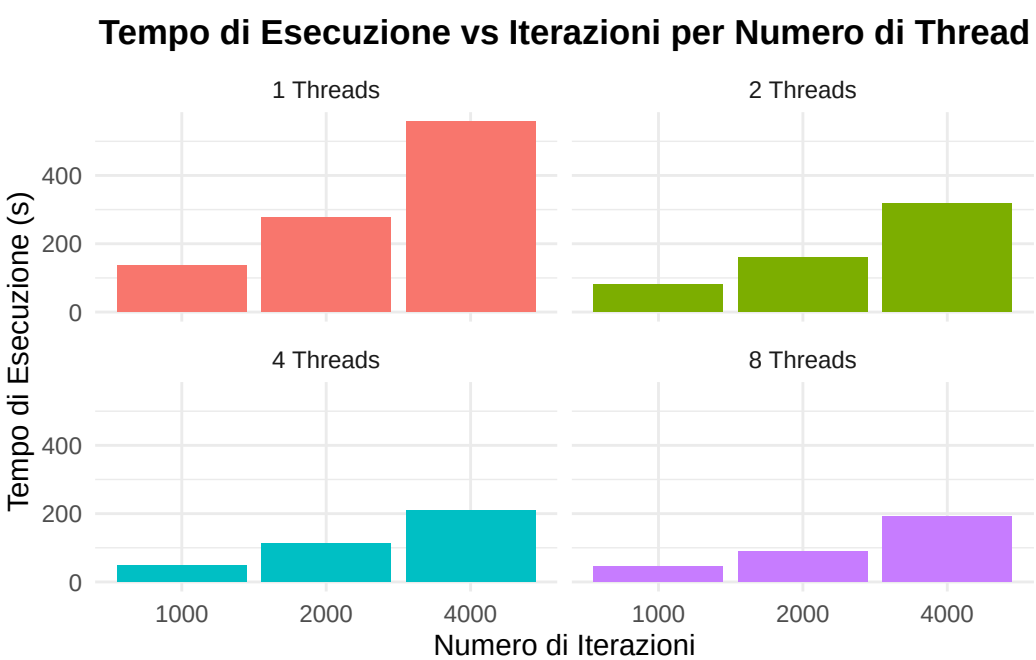
7. Colora il punto (a, b) in base al risultato di λ
8. Ripeti step (3-7) per ogni punto dell'immagine

3 OpenMP

i tipi di schedule non cambiano significamente il risultato dei test essendo che nel codice non ci sono condizioni di uscita preliminari quindi il programma fa fare alla cpu sempre lo stesso

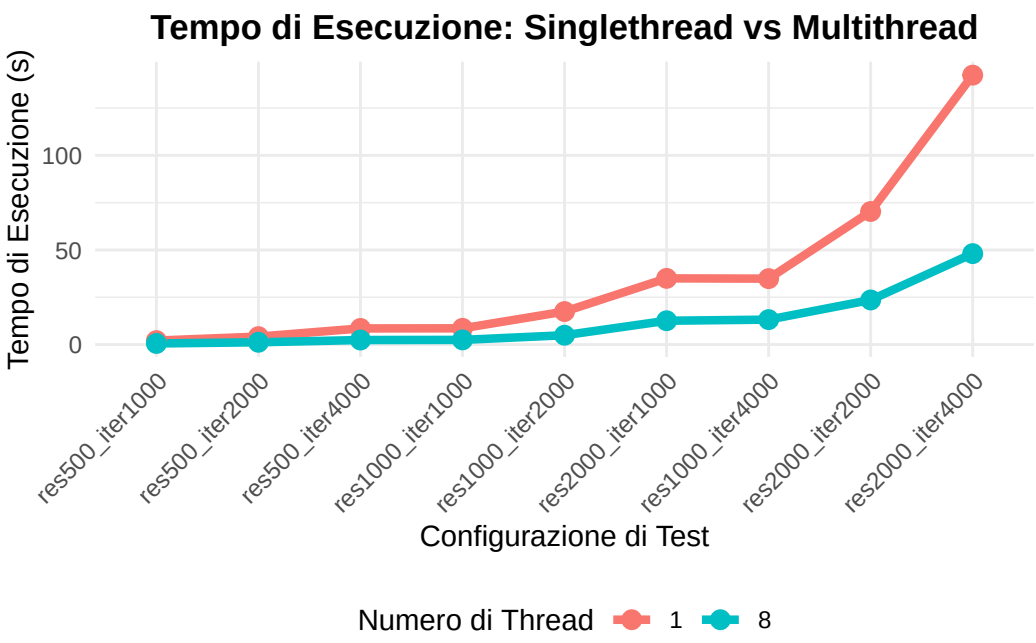
numero di calcoli indipendentemente dalle condizioni quindi ogni computazione richiede piu' o meno lo stesso tempo e cosi' la differenza fra tipi di schedule non e' tanta

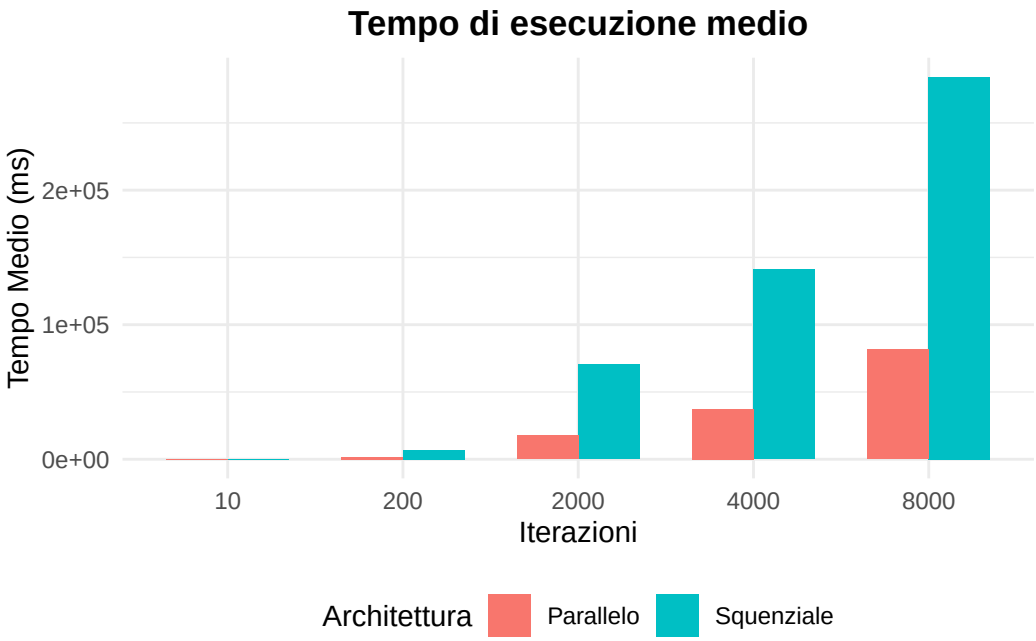
```
# Assicurati che il file .csv sia nella stessa cartella del report o fornisci il percorso
data_quad <- read.csv("benchmark_results_quad_core.csv")
data_quad %>%
  ggplot(aes(x = factor(Iterations), y = Time_ms / 1000, fill = factor(Threads))) +
  geom_col() +
  facet_wrap(~ paste(Threads, "Threads")) +
  labs(
    title = "Tempo di Esecuzione vs Iterazioni per Numero di Thread",
    x = "Numero di Iterazioni",
    y = "Tempo di Esecuzione (s)"
  ) +
  theme(legend.position = "none") # La legenda è superflua grazie a facet_wrap
```




```
data_quad %>%
  dplyr::filter(Threads %in% c(1, 8)) %>%
  group_by(Test_Name, Threads) %>%
  summarise(Mean_Time_s = mean(Time_ms) / 1000, .groups = 'drop') %>%
  # Usiamo 'color' per le linee/punti e specifichiamo 'group'
  ggplot(aes(x = reorder(Test_Name, Mean_Time_s), y = Mean_Time_s, color = factor(Threads)
```

```r
  geom_line(linewidth = 1.5) + # 'linewidth' è l'argomento più recente per lo spessore
  geom_point(size = 3) +
  labs(
    title = "Tempo di Esecuzione: Singlethread vs Multithread",
    x = "Configurazione di Test",
    y = "Tempo di Esecuzione (s)",
    color = "Numero di Thread" # Aggiorniamo l'etichetta della legenda
  ) +
theme(axis.text.x = element_text(angle = 45, hjust = 1))
```





3.1 Scheduling

```
df_dynamic <- read_csv("benchmark_results_dynamic.csv") %>% mutate(Schedule = "Dynamic")
```

```
Rows: 216 Columns: 7
-- Column specification --------------------------------------------------------
Delimiter: ","
chr (2): Test_Name, Sequence
dbl (5): Threads, Width, Height, Iterations, Time_ms

i Use `spec()` to retrieve the full column specification for this data.
i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```
df_guided  <- read_csv("benchmark_results_giuded.csv") %>% mutate(Schedule = "Guided")
```

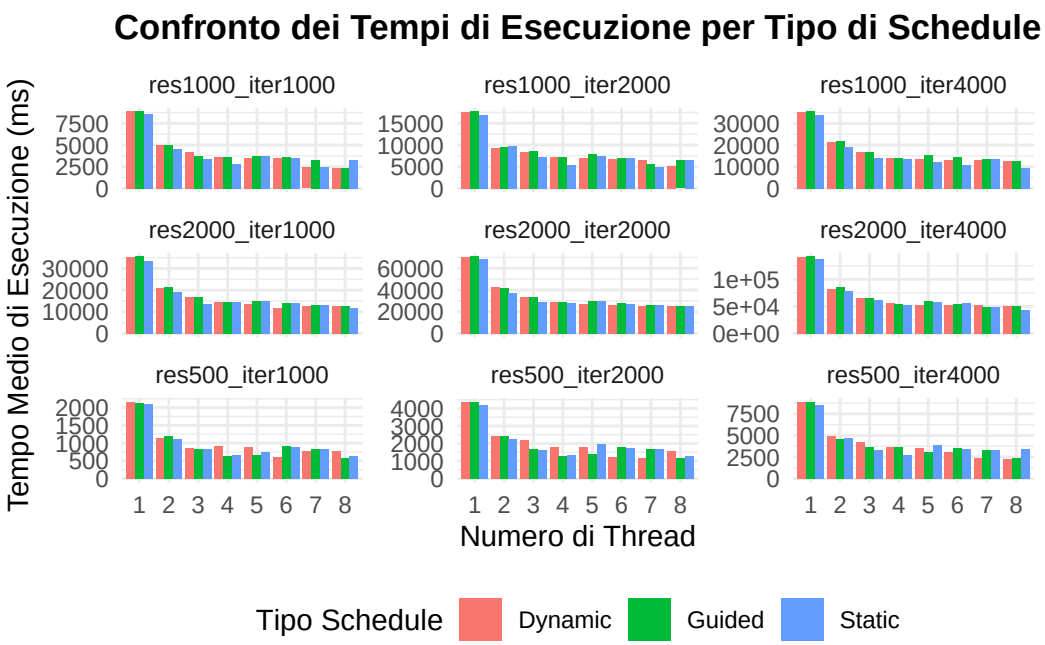
```
Rows: 216 Columns: 7
-- Column specification --------------------------------------------------------
Delimiter: ","
chr (2): Test_Name, Sequence
```

```
dbl (5): Threads, Width, Height, Iterations, Time_ms

i Use `spec()` to retrieve the full column specification for this data.
i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```r
df_static  <- read_csv("benchmark_results_static.csv") %>% mutate(Schedule = "Static")
```

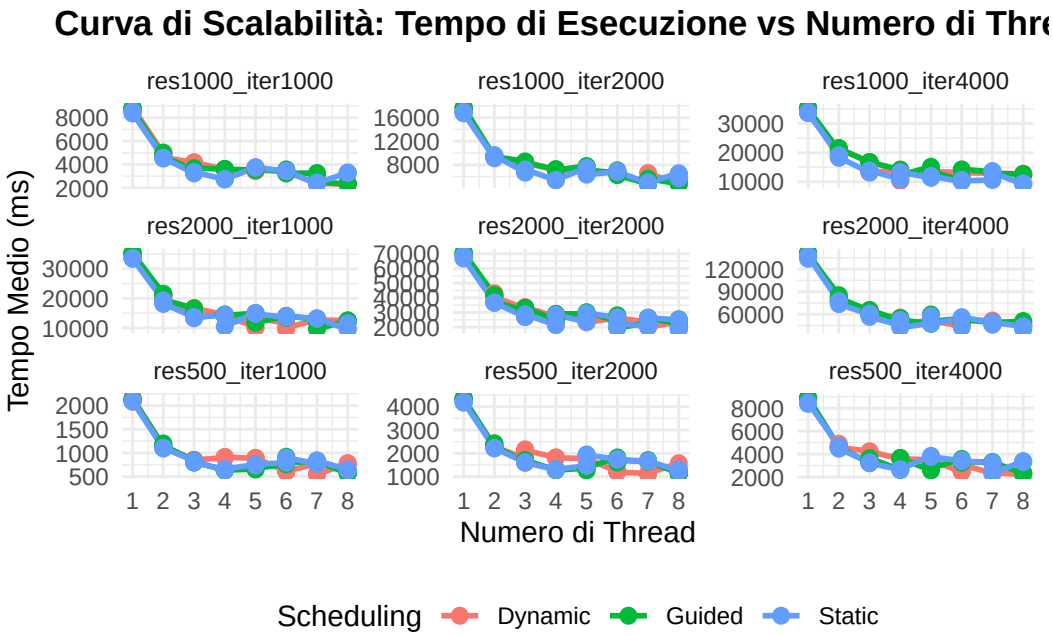
```
Rows: 216 Columns: 7
-- Column specification --------------------------------------------------------
Delimiter: ","
chr (2): Test_Name, Sequence
dbl (5): Threads, Width, Height, Iterations, Time_ms

i Use `spec()` to retrieve the full column specification for this data.
i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```r
df_combined <- bind_rows(df_dynamic, df_guided, df_static)

df_combined %>%
  ggplot(aes(x = factor(Threads), y = Time_ms, fill = Schedule)) +
  geom_bar(stat = "identity", position = "dodge") +
  facet_wrap(~ Test_Name, scales = "free_y") +
  labs(
    title = "Confronto dei Tempi di Esecuzione per Tipo di Schedule",
    x = "Numero di Thread",
    y = "Tempo Medio di Esecuzione (ms)",
    fill = "Tipo Schedule"
  )
```




```
df_combined %>%
  ggplot(aes(x = Threads, y = Time_ms, color = Schedule, group = Schedule)) +
  geom_line(size = 1) +
  geom_point(size = 2.5) +
  facet_wrap(~ Test_Name, scales = "free_y") +
  labs(
    title = "Curva di Scalabilità: Tempo di Esecuzione vs Numero di Thread",
    x = "Numero di Thread",
    y = "Tempo Medio (ms)",
    color = "Scheduling"
  ) +
  # Usiamo breaks dinamici per assicurarci che l'asse X mostri i thread interi (1,2,3,4...
  scale_x_continuous(breaks = unique(df_combined$Threads))
```

```
Warning: Using `size` aesthetic for lines was deprecated in ggplot2 3.4.0.
i Please use `linewidth` instead.
```



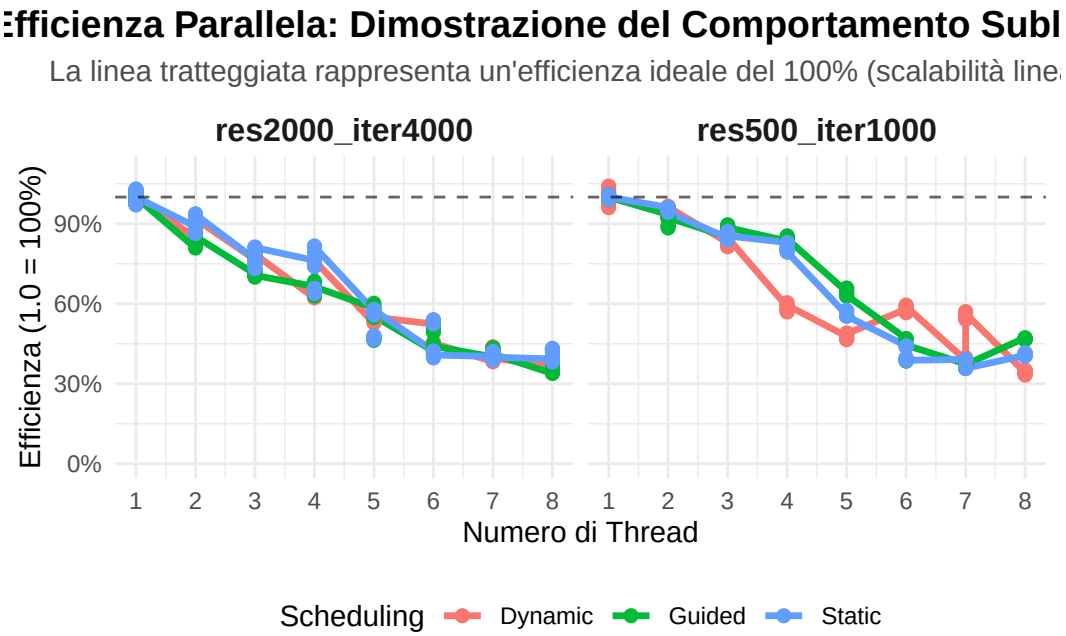
3.2 Sublinearita'

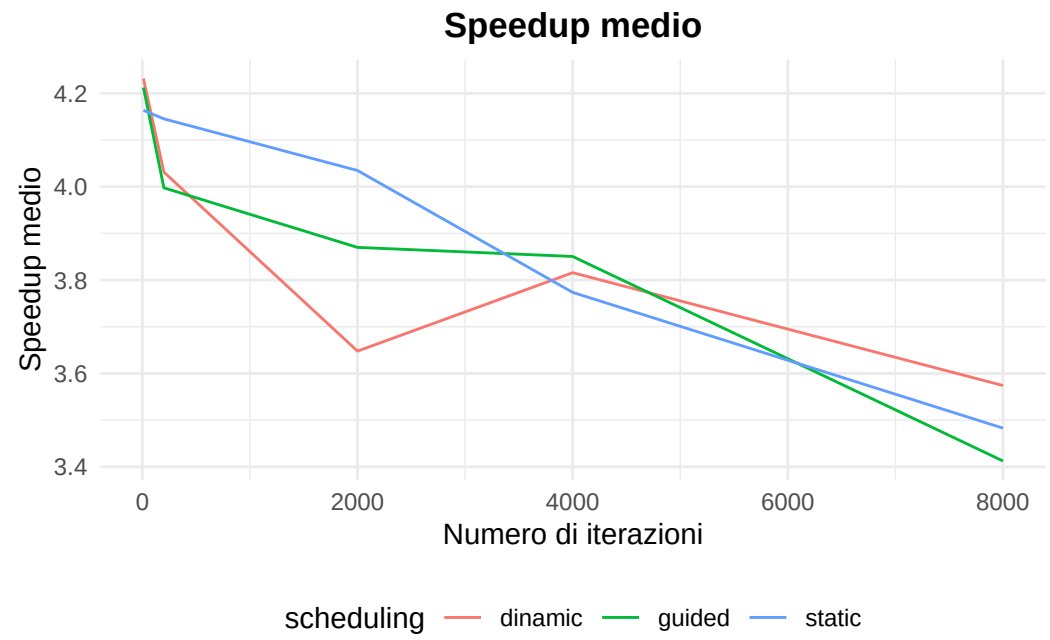
```
test_selezionati <- c("res500_iter1000", "res2000_iter4000")
df_filtered <- df_combined %>% filter(Test_Name %in% test_selezionati)

df_baseline <- df_combined %>%
  filter(Threads == 1) %>%
  select(Test_Name, Schedule, Base_Time = Time_ms)

# 4. Calcolo Speedup e **Efficienza**
df_metrics <- df_combined %>%
  left_join(df_baseline, by = c("Test_Name", "Schedule")) %>%
  mutate(
    Speedup = Base_Time / Time_ms,
    Efficiency = Speedup / Threads     # Questa è la formula chiave!
  )
```

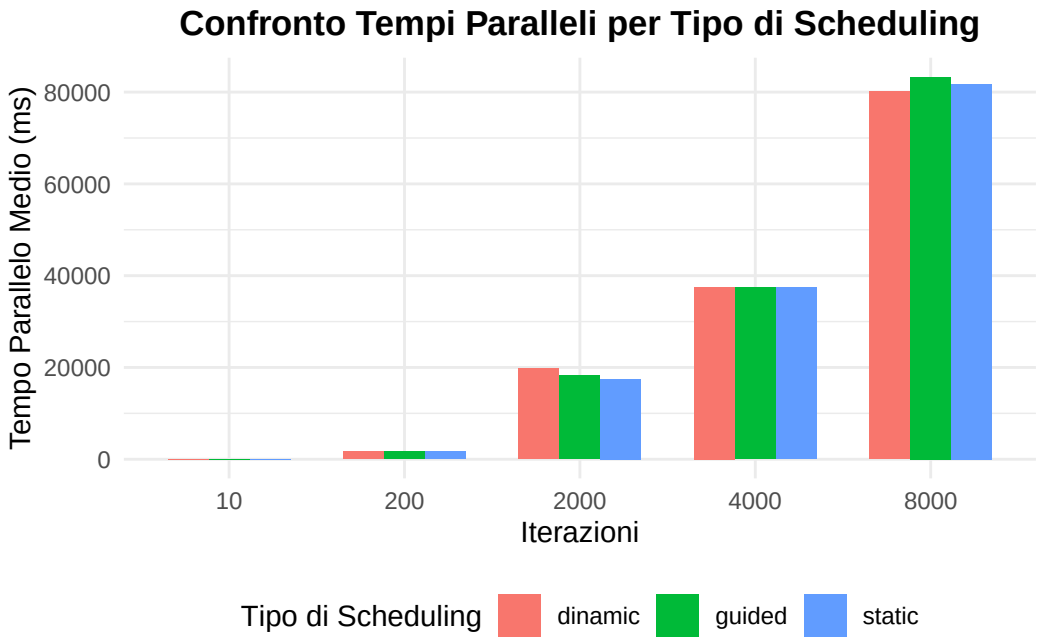
```
Warning in left_join(., df_baseline, by = c("Test_Name", "Schedule")): Detected an unexpecte
to-many relationship between `x` and `y`.
i Row 1 of `x` matches multiple rows in `y`.
i Row 1 of `y` matches multiple rows in `x`.
```

```
i If a many-to-many relationship is expected, set `relationship =
  "many-to-many"` to silence this warning.
```

```r
# 5. Creazione del grafico dell'Efficienza Parallela
df_metrics %>%
  dplyr::filter(Test_Name %in% test_selezionati) %>%
  ggplot(aes(x = Threads, y = Efficiency, color = Schedule, group = Schedule)) +
  geom_line(size = 1.2) +
  geom_point(size = 2) +

  # Linea orizzontale a Y=1 che rappresenta l'efficienza ideale (100% lineare)
  geom_hline(yintercept = 1, linetype = "dashed", color = "black", alpha = 0.6) +

  facet_wrap(~ Test_Name) +

  labs(
    title = "Efficienza Parallela: Dimostrazione del Comportamento Sublineare",
    subtitle = "La linea tratteggiata rappresenta un'efficienza ideale del 100% (scalabili
    x = "Numero di Thread",
    y = "Efficienza (1.0 = 100%)",
    color = "Scheduling"
  ) +
  # Forziamo l'asse X a mostrare solo i thread reali e l'asse Y da 0 a 1.1 per chiarezza
  scale_x_continuous(breaks = unique(df_metrics$Threads)) +
  scale_y_continuous(limits = c(0, 1.1), labels = scales::percent_format(accuracy = 1)) +
  theme_minimal() +
  theme(
    plot.title = element_text(hjust = 0.5, face = "bold", size = 14),
    plot.subtitle = element_text(hjust = 0.5, size = 11, color = "gray30"),
    strip.text = element_text(size = 12, face = "bold"),
    legend.position = "bottom"
  )
```

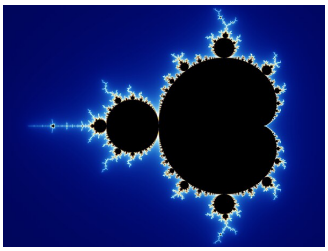

Efficienza Parallela: Dimostrazione del Comportamento Subl
La linea tratteggiata rappresenta un'efficienza ideale del 100% (scalabilità line
res2000_iter4000
res500_iter1000
Efficienza (1.0 = 100%)
90%
60%
30%
0%
1 2 3 4 5 6 7 8
Numero di Thread
Scheduling
Dynamic
Guided
Static

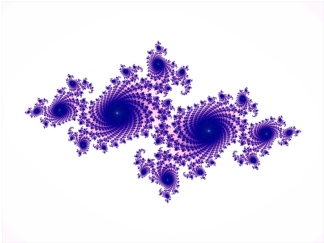
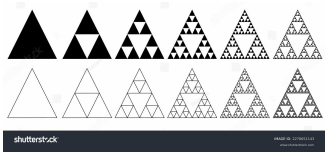
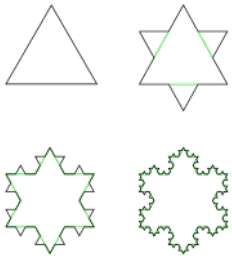
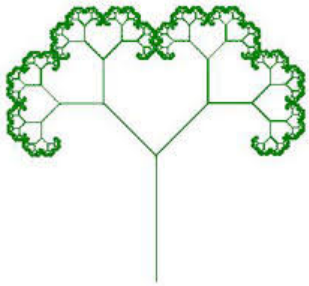
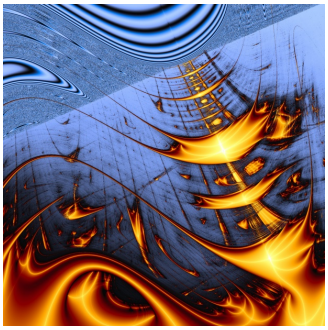

Speedup medio
Speedup medio
4.2
4.0
3.8
3.6
3.4
0
2000
4000
6000
8000
Numero di iterazioni
scheduling
dinamic
guided
static



i tipi di schedule non cambiano significamente il risultato dei test essendo che nel codice non ci sono condizioni di uscita preliminari quindi il programma fa fare alla cpu sempre lo stesso numero di calcoli indipendentemente dalle condizioni quindi ogni computazione richiede piu' o meno lo stesso tempo e cosi' la differenza fra tipi di schedule non e' tanta

Alcuni dei frattali piu' famosi sono:

Frattale	Definizione Matematica	Caratteristica	Immagine
Insieme di Mandelbrot	$z_{n+1} = z_n^2 + c$	Icona del confine tra ordine e caos.	

Frattale	Definizione Matematica	Caratteristica	Immagine
Insieme di Julia	$z_{n+1} = z_n^2 + c$	Varianti spettacolari di Mandelbrot.	
Triangolo di Sierpiński	Sottrazione ricorsiva	Geometria discreta autosimilare.	
Fiocco di Neve di Koch	Triangolazione lati	Perimetro infinito, area finita.	
Felce di Barnsley	IFS	Algoritmi biologici frattali.	

Frattale	Definizione Matematica	Caratteristica	Immagine
Frattale di Lyapunov	Esponente λ	Stabilità in sistemi dinamici.